GROUP 14

4-bit Carry Look-Ahead Adder v/s Ripple Carry Adder

PRESENTED BY 15 DILJITH OR 47 PETER AJITH CHERIAN

INTRODUCTION

4 bit carry look-ahead adder is a type of adder used in digital circuits to add binary numbers **fast and efficiently** .

Unlike a ripple carry adder, it speeds up the process by **calculating carries in advance.**

This is a fundamental concept in **designing high speed arithmetic logic units** (ALUs) in modern processors.

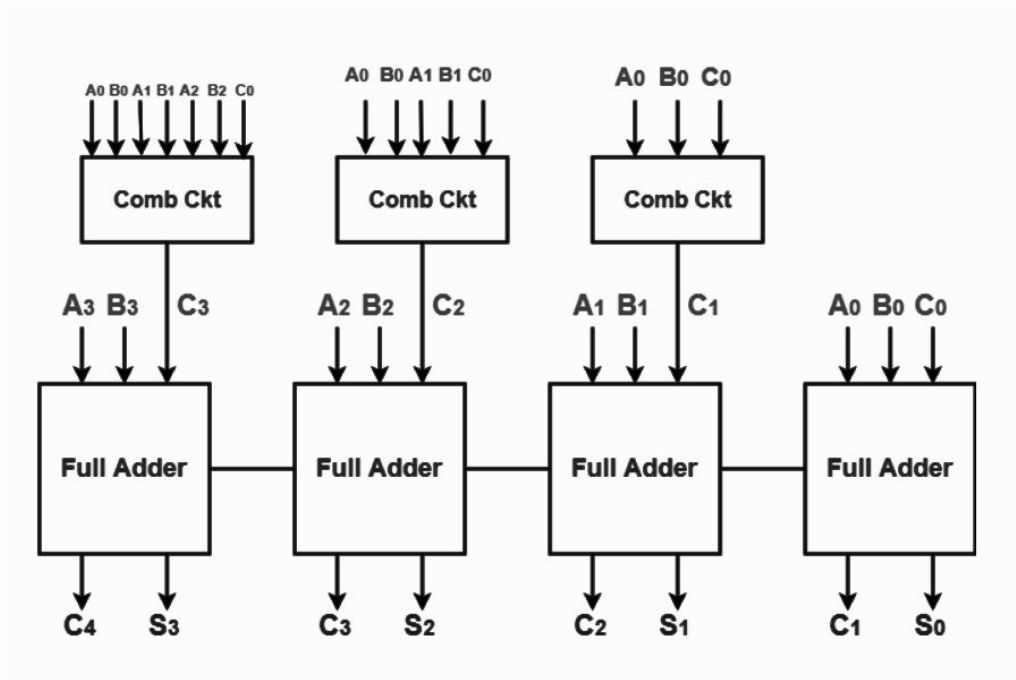
OBJECTIVES

- Design a **4-bit Carry Look-Ahead Adder (CLA)** using Verilog.
- Compare its performance with a **Ripple Carry Adder (RCA)**.
- Implement both designs on an FPGA.
- Measure key performance metrics such as **delay** and **power consumption**.

SCOPE

- **Design** : Implement CLA and RCA adders using Verilog HDL.
- **Test** : Simulate both designs with Icarus Verilog.
- **Implement**: Deploy on an FPGA for real-world performance testing.
- **Analyze** : Compare propagation delay, area, and power consumption.
- Suitable for high-performance systems like CPUs, DSPs, and microcontrollers, where fast arithmetic operations are essential.

BLOCK DIAGRAM



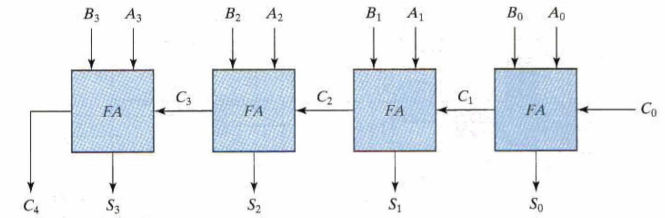


FIGURE 4.9
Four-bit adder

four-bit binary ripple carry adder.

Example.

Consider the two binary numbers A = 1011 and B = 0011.

Their sum S = 1110 is formed with the four-bit adder as :

Subscript i:	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

Four-bit Full Adder with Carry Lookahead

Carry look ahead logic.

C_0 = input carry

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

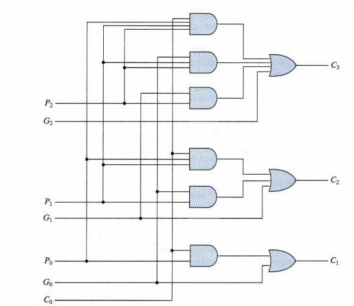


FIGURE 4.11
Logic diagram of carry lookahead generator

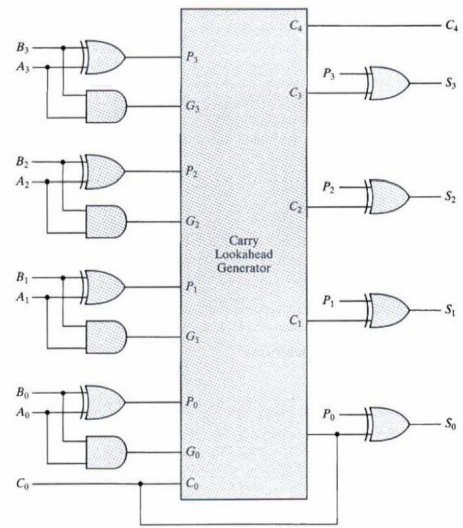


FIGURE 4.12
Four-bit adder with carry lookahead

DESIGN

1. 4-bit Ripple Carry Adder (RCA)

This design uses a series of 1-bit full adders.

1-bit Full Adder Code

```
module full_adder (
    input A, B, Cin,
    output Sum, Cout
);
    assign Sum = A ^ B ^ Cin;         // Sum = A XOR B XOR Cin
    assign Cout = (A & B) | (Cin & (A ^ B)); // Cout = AB + Cin(A XOR B)
endmodule
```

4-bit Ripple Carry Adder Code

```
module ripple_carry_adder_4bit (
    input [3:0] A, B,
    input Cin,
    output [3:0] Sum,
    output Cout
);
    wire C1, C2, C3;  // Internal carry wires

    // Instantiate 1-bit full adders
    full_adder FA0(A[0], B[0], Cin, Sum[0], C1);
    full_adder FA1(A[1], B[1], C1, Sum[1], C2);
    full_adder FA2(A[2], B[2], C2, Sum[2], C3);
    full_adder FA3(A[3], B[3], C3, Sum[3], Cout);
endmodule
```

2. 4-bit Carry Look-Ahead Adder (CLA)

```
module carry_lookahead_adder_4bit (
    input [3:0] A, B,
    input Cin,
    output [3:0] Sum,
    output Cout
);
    wire [3:0] G, P;     // Generate and Propagate signals
    wire C1, C2, C3;     // Internal carry signals

    // Generate and Propagate Logic
    assign G = A & B;    // Generate: Gi = Ai & Bi
    assign P = A | B;    // Propagate: Pi = Ai | Bi

    // Carry Look-Ahead Logic
    assign C1 = G[0] | (P[0] & Cin);  // C1 = G0 + P0 * Cin
    assign C2 = G[1] | (P[1] & C1);   // C2 = G1 + P1 * C1
    assign C3 = G[2] | (P[2] & C2);   // C3 = G2 + P2 * C2
    assign Cout = G[3] | (P[3] & C3); // Cout = G3 + P3 * C3

    // Sum Calculation
    assign Sum[0] = A[0] ^ B[0] ^ Cin;  // Sum0 = A0 XOR B0 XOR Cin
    assign Sum[1] = A[1] ^ B[1] ^ C1;   // Sum1 = A1 XOR B1 XOR C1
    assign Sum[2] = A[2] ^ B[2] ^ C2;   // Sum2 = A2 XOR B2 XOR C2
    assign Sum[3] = A[3] ^ B[3] ^ C3;   // Sum3 = A3 XOR B3 XOR C3
endmodule
```

3. Testbench for Both Adders

```
module testbench();
    reg [3:0] A, B;
    reg Cin;
    wire [3:0] Sum_RCA, Sum_CLA;
    wire Cout_RCA, Cout_CLA;
  // Instantiate the RCA
    ripple_carry_adder_4bit RCA (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum_RCA),
        .Cout(Cout_RCA)
    );
    // Instantiate the CLA
    carry_lookahead_adder_4bit CLA (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum_CLA),
        .Cout(Cout_CLA)
    );
    initial begin
        // Test cases
        A = 4'b0000; B = 4'b0000; Cin = 0; // Test Case 1
        #10; // Wait for 10 time units

        A = 4'b0011; B = 4'b0001; Cin = 0; // Test Case 2
        #10;

        A = 4'b0110; B = 4'b0101; Cin = 1; // Test Case 3
        #10;

        A = 4'b1111; B = 4'b1111; Cin = 1; // Test Case 4
        #10;
        // End simulation
        $finish;
    end

    initial begin
        // Monitor outputs
        $monitor("A = %b, B = %b, Cin = %b | Sum_RCA = %b, Cout_RCA = %b | Sum_CLA = %b, Cout_CLA = %b",
                 A, B, Cin, Sum_RCA, Cout_RCA, Sum_CLA, Cout_CLA);
    end
endmodule
```

SOFTWARE/HARDWARE REQUIREMENTS

Software:

- Icarus Verilog: Used for simulation and verification of Verilog designs.
  - Verilog files for CLA and RCA.
  - Testbenches to verify functionality.
- GTKWave: To view simulation waveforms and analyze timing.

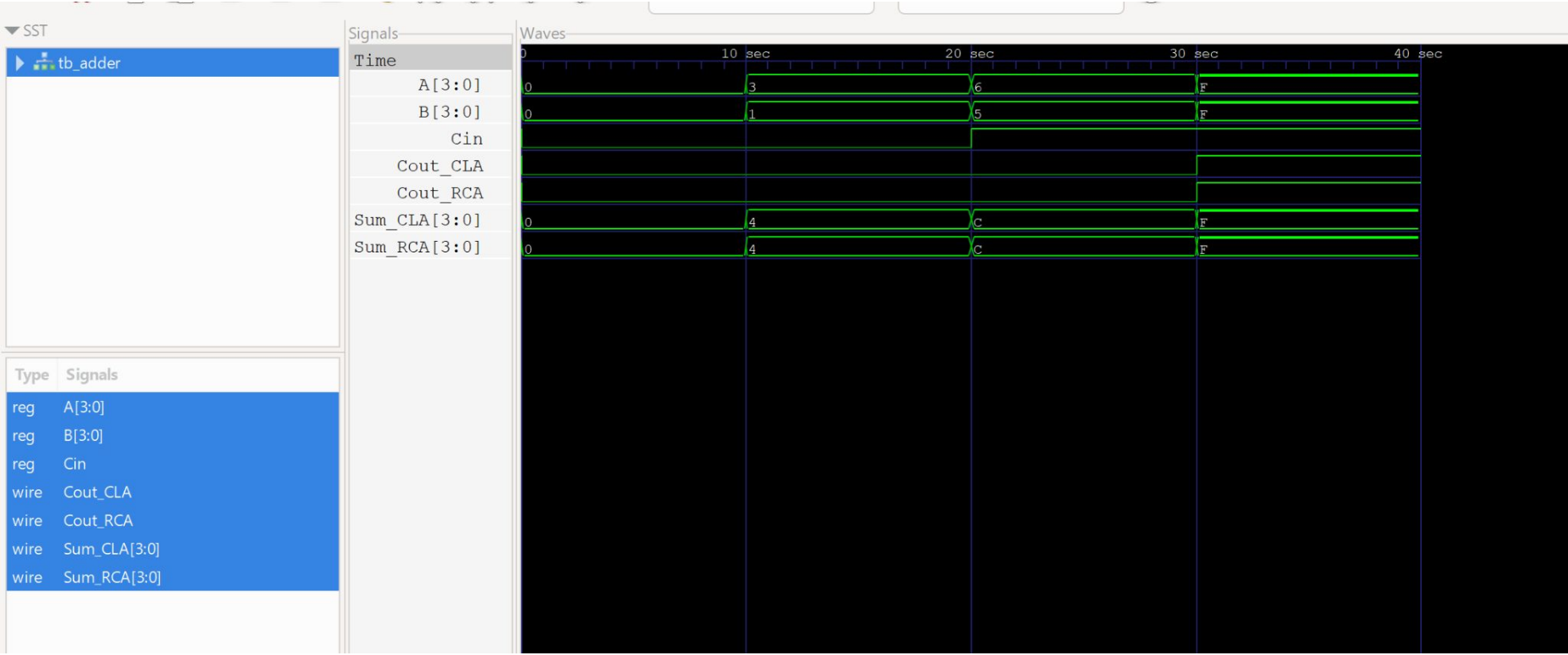
Hardware:

- FPGA development board (Xilinx FPGA).
- Synthesis Tools: Xilinx Vivado for FPGA deployment.

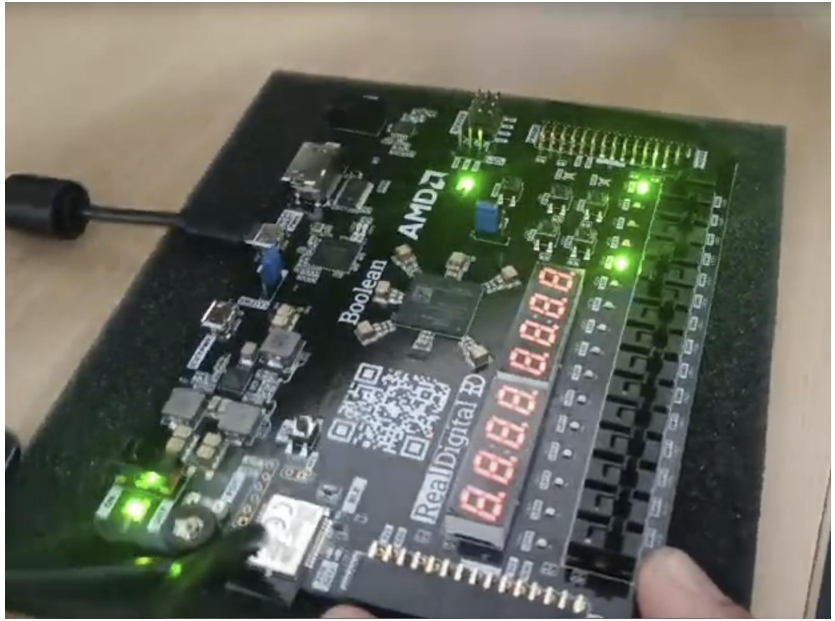
BILL OF MATERIALS (BOM)

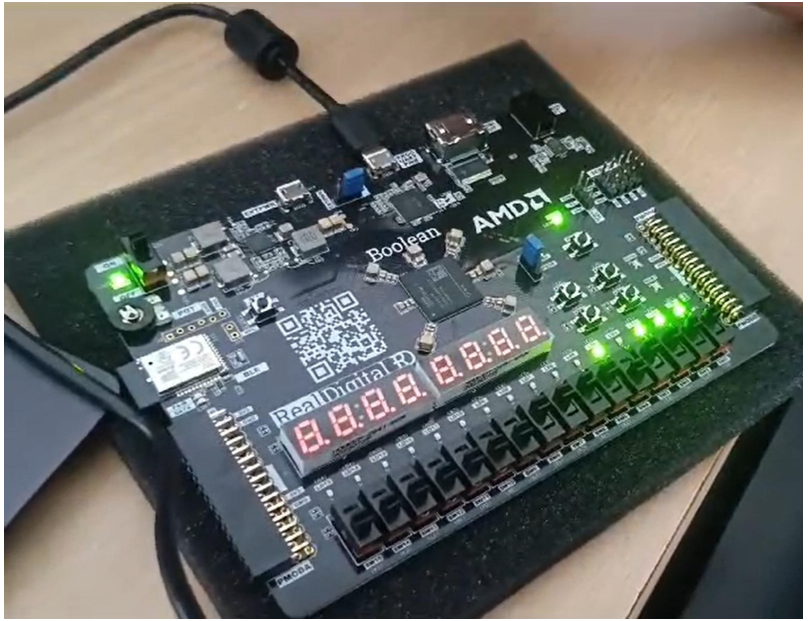
- **FPGA Development Board (Xilinix)**: Used for hardware implementation.
- **Icarus Verilog**: Open-source Verilog simulator.
- **HDL Design Tools (Vivado)**: For synthesis and deployment.
- **Power Supply & Peripheral Devices:** Required for the FPGA board.

SIMULATION O/P



FPGA IMPLEMENTATION





CONCLUSION

This project effectively implements a 4-bit Carry Look-Ahead Adder using Verilog. The 4-bit Carry Look-Ahead Adder offers faster addition by reducing the delay caused by ripple carry propagation.

Tests and adjustments were carried out to verify the correct operation of the 4-bit Carry Look-Ahead Adder, and the results produced by the code have been demonstrated.

The 4 Bit Carry Look-Ahead Adder significantly improves addition speed by minimizing carry propagation delays, making it ideal for high-performance computing tasks.

REFERENCES

- **Digital Design** by Morris Mano
- https://www.gatevidyalay.com/carry-look-ahead-adder/ (Block diagram)
- https://stackoverflow.com/questions/68964488/verilog-test-bench-for-4-bit-adder-with-carry-lookahead (Test Bench)
- https://nandland.com/carry-lookahead-adder/ (ideas)
- https://github.com/roboticvedant/CLA_4bit_adder